

Redes Neuronales Artificiales - Parte Uno

Técnicas de Inteligencia Artificial

Prof. Flavio Prieto
email: faprieto@unal.edu.co

INGENIERÍA MECATRONICA
Facultad de Ingeniería
Universidad Nacional de Colombia Sede Bogotá



19 de febrero de 2026

Orígenes de las Redes Neuronales Artificiales.

- ▶ En la década de 1940, McCulloch y Pitts (1943) propusieron el primer modelo matemático de neurona artificial, inspirado en la lógica binaria .

A LOGICAL CALCULUS OF THE IDEAS IMMANENT IN NERVOUS ACTIVITY

WARREN S. MCCULLOCH and WALTER H. PITTS

Fuente: [Ref.](#)

- ▶ El modelo se basa en la siguiente ecuación:

$$y = H \left(\sum_i w_i x_i - \theta \right)$$

donde:

- ▶ y es la salida del modelo.
 - ▶ $H(\cdot)$ es la función escalón, que define la activación del modelo.
 - ▶ w_i es el peso asociado a la entrada x_i .
 - ▶ x_i es la i -ésima característica o variable de entrada.
 - ▶ $\sum_i w_i x_i$ representa la combinación lineal ponderada de las entradas.
 - ▶ θ es el umbral (bias) que determina el punto de activación.
-
- ▶ Este modelo mostró cómo redes compuestas por estas unidades pueden implementar operaciones lógicas y funciones computables.

1958–1960: Rosenblatt y el Perceptrón.

- ▶ Frank Rosenblatt diseña **el perceptrón**, un **modelo entrenable de neurona** artificial.
- ▶ En 1958 construye la máquina *Mark I Perceptrón*, orientada al reconocimiento de imágenes.

Psychological Review
Vol. 65, No. 6, 1958

THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN¹

F. ROSENBLATT

Cornell Aeronautical Laboratory

Fuente: [Ref.](#)

- ▶ El modelo está compuesto por una capa sensorial, una capa asociativa y una capa de respuesta.
- ▶ Utiliza **pesos ajustables** mediante un algoritmo de aprendizaje supervisado:

$$w_i \leftarrow w_i + \eta(y - \hat{y})x_i$$

donde:

- ▶ η es la tasa de aprendizaje,
 - ▶ y la salida esperada y
 - ▶ $\hat{y}$ la salida predicha.
-
- ▶ Rosenblatt demuestra éxito del perceptrón en tareas simples como la clasificación entre “X” y “E”, o entre cuadrados y diamantes.

1969–1970: Minsky & Papert y la “Primera Crisis”.

- ▶ Marvin Minsky y Seymour Papert publican en 1969 el libro *Perceptrons*¹, en el que demuestran que **los perceptrones simples no pueden resolver funciones no linealmente separables**, como la función XOR.
- ▶ En su análisis, comparan la capacidad expresiva de los perceptrones y resaltan limitaciones fundamentales del modelo.
- ▶ **Esta crítica técnica tuvo un fuerte impacto**: frenó la financiación de proyectos relacionados con redes neuronales.
- ▶ El escepticismo resultante condujo al **primer “invierno de la inteligencia artificial”** (*AI winter*), marcado por una notable disminución en el interés y el apoyo institucional.

¹Minsky, Marvin; Papert, Seymour (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press. ISBN 978-0-262-63022-1.

Años 1980: Resurgimiento con Backpropagation.

- ▶ En 1986, Rumelhart² y Williams **introducen el algoritmo de retropropagación** para el entrenamiento de redes neuronales multicapa.
- ▶ El algoritmo se basa en la **regla de la cadena** para calcular los gradientes de la función de pérdida y actualizar los pesos sinápticos de forma eficiente.

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton†
& Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California,
San Diego, La Jolla, California 92093, USA

† Department of Computer Science, Carnegie-Mellon University,
Pittsburgh, Philadelphia 15213, USA

Fuente: [Ref.](#)

²Rumelhart, Hinton & Williams (1986): Learning representations by back-propagating errors, publicado en Nature, Vol. 323, pp. 533–536, el 9 de octubre de 1986

- ▶ En 1989, Yann LeCun **aplica la retropropagación en redes convolucionales** (LeNet)³, logrando reconocimiento automático de dígitos manuscritos en cheques bancarios.

Backpropagation Applied to Handwritten Zip Code Recognition

Y. LeCun
B. Boser
J. S. Denker
D. Henderson
R. E. Howard
W. Hubbard
L. D. Jackel

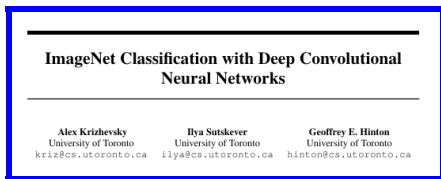
AT&T Bell Laboratories, Holmdel, NJ 07733 USA

Fuente: [Ref.](#)

³Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, *et al.* Backpropagation Applied to Handwritten Zip Code Recognition. Neural Computation, Vol. 1, No. 4 (invierno de 1989).

Década de 2010: Auge del Deep Learning.

- ▶ En 2012, AlexNet⁴ (Krizhevsky, Sutskever y Hinton) revoluciona la visión computacional al introducir una **red convolucional profunda** con:
 - ▶ 8 capas, aproximadamente 60 millones de parámetros, funciones de activación ReLU, entrenamiento acelerado por GPU, y regularización mediante *dropout*.
- ▶ AlexNet gana el concurso ILSVRC en septiembre de 2012 con un error *top-5* del 15.3 %, marcando el **inicio de la “primavera de la inteligencia artificial”**.

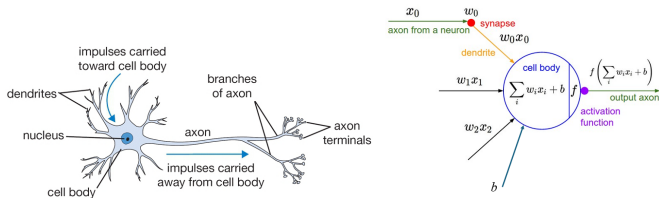


Fuente: [Ref.](#)

⁴Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet Classification with Deep Convolutional Neural Networks, NeurIPS 2012.

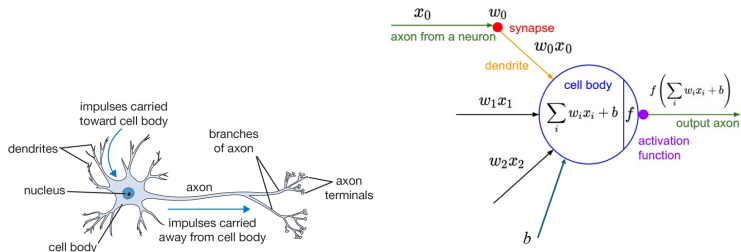
Motivación Biológica y Conexiones.

- ▶ La unidad computacional básica del cerebro es la neurona. El sistema nervioso humano contiene aproximadamente **86 mil millones de neuronas**, conectadas por entre 10^{14} y 10^{15} sinapsis.
- ▶ La figura ilustra una neurona biológica (izquierda) y su modelo computacional simplificado (derecha).



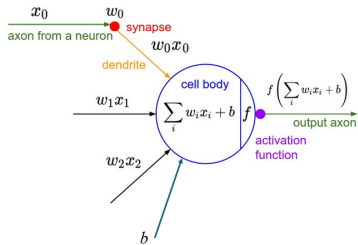
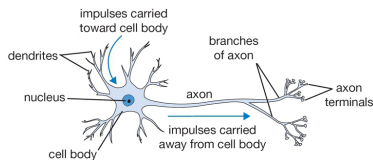
Fuente: [Internet](#).

Motivación Biológica y Conexiones



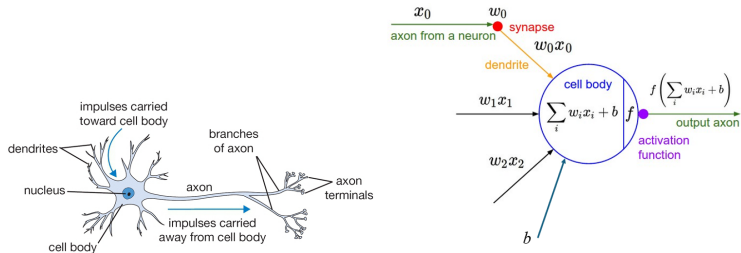
- ▶ Cada neurona **recibe señales de entrada** a través de sus dendritas y produce señales de salida a través de un único axón.
- ▶ El axón se ramifica y se conecta, mediante **sinapsis**, con las dendritas de otras neuronas.
 - ▶ En el modelo computacional, las señales que viajan por los axones (por ejemplo, x_0) interactúan de forma multiplicativa con las dendritas receptoras mediante un peso sináptico (por ejemplo, w_0x_0).

Motivación Biológica y Conexiones



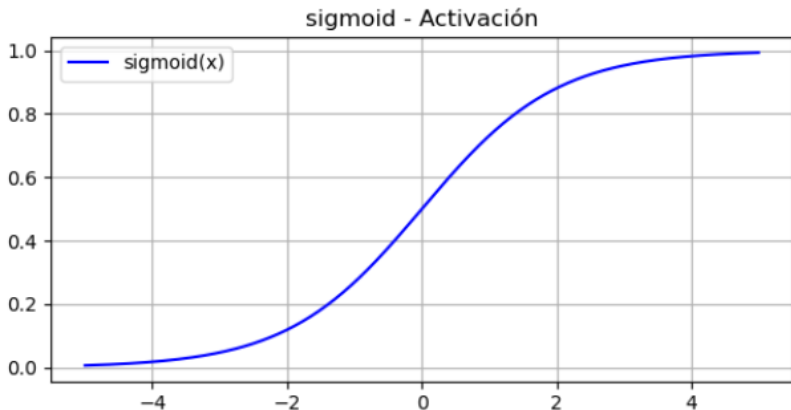
- ▶ Los pesos sinápticos w son **parámetros ajustables** que determinan la fuerza (y dirección: excitatoria si es positiva o inhibitoria si es negativa) de influencia entre neuronas.
- ▶ El sesgo b es un **parámetro ajustable** que desplaza la función de activación, permitiendo fijar el umbral de disparo de la neurona de manera independiente de las entradas.

Motivación Biológica y Conexiones



- ▶ En el modelo básico, **las señales recibidas por las dendritas se suman** en el cuerpo celular.
 - ▶ **Si esta suma supera un umbral, la neurona se activa** y genera un impulso a lo largo de su axón.
- ▶ En el modelo computacional, se asume que no importa el tiempo exacto del disparo, sino la frecuencia.
 - ▶ Esta frecuencia se modela mediante una función de activación f , que representa la tasa de disparo de la neurona.

- ▶ Históricamente, una función de activación común ha sido la **sigmoide** σ , que toma una entrada real y la convierte en un valor entre 0 y 1, representando así una tasa de disparo normalizada.



El perceptrón.

- ▶ El perceptrón fue propuesto por Frank Rosenblatt en 1958 como un modelo computacional inspirado en las neuronas biológicas.
- ▶ Se trata de un clasificador binario supervisado capaz de aprender una frontera de decisión lineal.
- ▶ Representa la base de las redes neuronales artificiales y del aprendizaje profundo.

Modelo Matemático del Perceptrón.

- ▶ Sea $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ el vector de entrada.
Valores numéricos que representan las características del dato a clasificar.
- ▶ Sea $\mathbf{w} = [w_1, w_2, \dots, w_n]^T$ el vector de pesos sinápticos.
Valores numéricos asociados a cada entrada, que indican la importancia de la entrada.
- ▶ **Suma Ponderada (Entrada Neta):** Se calcula la suma de las entradas multiplicadas por sus respectivos pesos, más el sesgo. A esta suma se le denomina entrada neta (z).

$$z = (w_1x_1 + w_2x_2 + \dots + w_nx_n) + b = \sum_{i=1}^n w_ix_i + b$$

O en notación vectorial: $z = \mathbf{w}^T \cdot \mathbf{x} + b$.

Modelo Matemático del Perceptrón (Cont.).

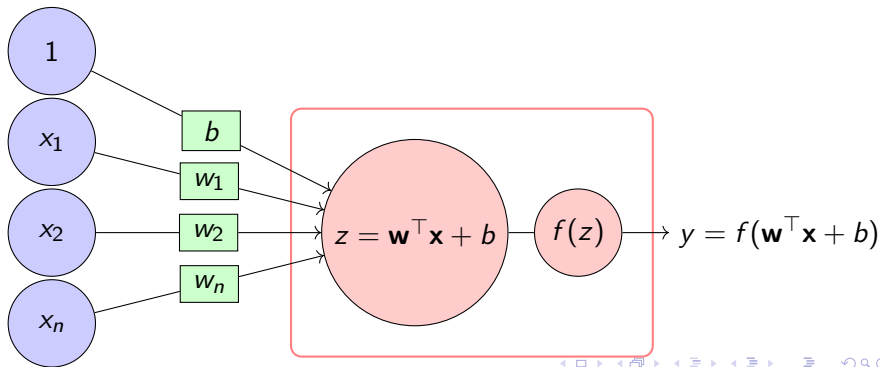
- ▶ La entrada neta z se pasa a través de una función de activación para producir la salida final $\hat{y}$.
El Perceptrón clásico utiliza una **función escalón (step function)**:

$$y = f(z) = f(\mathbf{w}^\top \mathbf{x} + b) = \begin{cases} 1, & \text{si } \mathbf{w}^\top \mathbf{x} + b \geq 0 \\ 0, & \text{en otro caso} \end{cases}$$

- ▶ Donde b (o w_0) es el sesgo (bias).
Un término adicional que permite ajustar el umbral de activación de la neurona.
- ▶ A menudo se representa como un peso w_0 con una entrada fija $x_0 = 1$.

$$z = (w_1x_1 + w_2x_2 + \cdots + w_nx_n) + b = \sum_{i=1}^n w_ix_i + b = \mathbf{w} \cdot \mathbf{x} + b$$

$$y = f(z) = f(\mathbf{w}^\top \mathbf{x} + b) = \begin{cases} 1, & \text{si } \mathbf{w}^\top \mathbf{x} + b \geq 0 \\ 0, & \text{en otro caso} \end{cases}$$



Funcionamiento del Perceptrón.

1. Recibe un vector de entrada $\mathbf{x}$.
2. Calcula una combinación lineal ponderada: $z = \mathbf{w}^T \mathbf{x} + b$.
3. Aplica la función de activación escalón para determinar la clase:

$$y = f(z)$$

4. Clasifica la entrada como una de dos clases (por ejemplo, 0 o 1).

Observación: El perceptrón solo puede separar clases que sean linealmente separables.

Ejemplo en Python: El perceptrón

Objetivo

- ▶ Comprender el funcionamiento del Perceptrón.
- ▶ **Ver Notebook.**

Algoritmo de Aprendizaje del Perceptrón.

Dado un conjunto de entrenamiento $\{(\mathbf{x}^{(i)}, y^{(i)})\}$, se actualizan los pesos según el error cometido:

$$w_j \leftarrow w_j + \eta(y^{(i)} - \hat{y}^{(i)})x_j^{(i)}$$

$$b \leftarrow b + \eta(y^{(i)} - \hat{y}^{(i)})$$

- ▶ $\eta > 0$ es la tasa de aprendizaje.
- ▶ $y^{(i)}$ es la etiqueta verdadera.
- ▶ $\hat{y}^{(i)}$ es la predicción.
- ▶ La actualización se realiza si hay error de clasificación.

Ejemplo Numérico Simple.

Función de activación escalón.

- ▶ Supóngase una entrada $\mathbf{x} = [1, 2]$, pesos iniciales $\mathbf{w} = [0,5, -0,5]$, $b = 0$, y $y = 1$.
- ▶ Cálculo: $z = 0,5 \cdot 1 + (-0,5) \cdot 2 = 0,5 - 1 = -0,5$
- ▶ Salida: $f(z) = 0$
- ▶ Error: $y^{(i)} - \hat{y}^{(i)} = 1 - 0 = 1$
- ▶ Actualización con $\eta = 1$:

$$w_1 \leftarrow 0,5 + 1 \cdot 1 \cdot 1 = 1,5,$$

$$w_2 \leftarrow -0,5 + 1 \cdot 1 \cdot 2 = 1,5$$

$$b \leftarrow 0 + 1 \cdot 1 = 1$$

Ejemplo Numérico con 5 Entradas.

Función de activación escalón.

- ▶ Supóngase una entrada $\mathbf{x} = [1, 0, -1, 2, 1]$, pesos iniciales $\mathbf{w} = [0,5, -0,5, 0, 1, -1]$, $b = 0$, y $y = 1$.

- ▶ Cálculo:

$$z = 0,5 \cdot 1 + (-0,5) \cdot 0 + 0 \cdot (-1) + 1 \cdot 2 + (-1) \cdot 1 = 0,5 + 0 + 0 + 2 - 1 = 1,5$$

- ▶ Salida: $f(z) = 1 \Rightarrow$ error: $1 - 1 = 0$
- ▶ No hay actualización porque el error es cero.

Ejemplo en Python: Algoritmo de aprendizaje del perceptrón

Objetivo

- ▶ Aplicar la regla de perceptrón actualizando pesos y sesgo solo si hay error, tal como:

$$w_j \leftarrow w_j + \eta(y^{(i)} - \hat{y}^{(i)})x_j^{(i)}, \quad b \leftarrow b + \eta(y^{(i)} - \hat{y}^{(i)})$$

- ▶ **Ver Notebook.**

El Perceptrón como Clasificador.

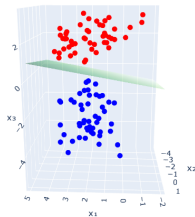
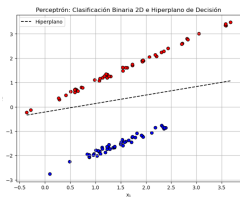
- ▶ El perceptrón es una neurona artificial que realiza una **clasificación binaria**.
- ▶ Su salida se define como:

$$y = \begin{cases} 1 & \text{si } \mathbf{w}^T \mathbf{x} + b \geq 0 \\ 0 & \text{en otro caso} \end{cases}$$

- ▶ El modelo aprende un **hiperplano de separación** en el espacio de características.

► Hiperplanos de separación en dos y tres dimensiones.

Perceptrón: Clasificación Binaria 3D e Hiperplano de Decisión



- Clase -1
- Clase +1

Ver código python.

Función de Costo del Perceptrón.

- ▶ El perceptrón busca separar los datos mediante un hiperplano definido por $\mathbf{w}$ y b .
- ▶ Se define una función de costo que se activa solo cuando un ejemplo es mal clasificado:

$$\mathcal{L}(\mathbf{w}, b; \mathbf{x}, y) = \max(0, -y(\mathbf{w}^\top \mathbf{x} + b)) = \max(0, -yz)$$

- ▶ Esta es una función por partes (hinge-like), donde:
 - ▶ Si $y(\mathbf{w}^\top \mathbf{x} + b) > 0$: no hay pérdida.
 - ▶ Si $y(\mathbf{w}^\top \mathbf{x} + b) \leq 0$: hay penalización.

La función $\mathcal{L}_{\text{hinge}} = \max(0, 1 - yz)$ es utilizada en el entrenamiento clásico del perceptrón.

Función de costo Hinge: Clasificador SVM Binario.

- ▶ La función pérdida hinge⁵ (bisagra), típica de SVM:

$$\mathcal{L}_{\text{hinge}} = \max(0, 1 - yz)$$

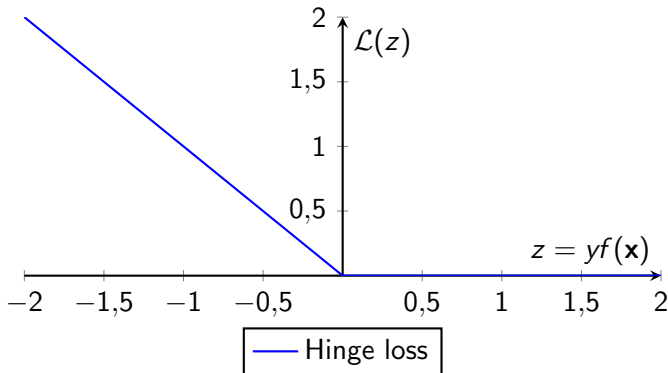
Interpretación:

- ▶ $z = \mathbf{w}^T \mathbf{x} + b$ es la salida cruda del modelo (sin función sigmoide).
- ▶ $y \in \{-1, 1\}$ es la etiqueta real.
- ▶ Si $(y z) \geq 1$: la pérdida es 0, $\Rightarrow$ Correctamente clasificado y *bien separado*.
- ▶ Si $0 < (y z) < 1$: $\mathcal{L} = 1 - (y z)$. $\Rightarrow$ Clasificado correctamente, pero *dentro del margen*; penalización proporcional a la distancia al margen.
- ▶ Si $(y z) \leq 0$: $\mathcal{L} = 1 - (y z)$. $\Rightarrow$ Clasificado *incorrectamente*; penalización linealmente creciente con la magnitud del error.
- ▶ La neurona se entrena con esta pérdida para emular un clasificador SVM (maximizar el margen entre las clases).

⁵Ver: [Sitio WEB](#).

- Visualmente, la superficie es plana en regiones sin error y con pendientes en zonas con errores de clasificación.

Gráfica de la Función Hinge.



Minimización: Descenso de Gradiente.

- ▶ La función de costo (o función de pérdida) del perceptrón es **no diferenciable** en ciertos puntos.
- ▶ Se utiliza una estrategia **de descenso de gradiente subestimado** (subgradiente).

La función de costo (o función de pérdida) se minimiza mediante un enfoque iterativo:

$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$$

- ▶ Para un ejemplo con error:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = -y\mathbf{x}, \quad \frac{\partial \mathcal{L}}{\partial b} = -y$$

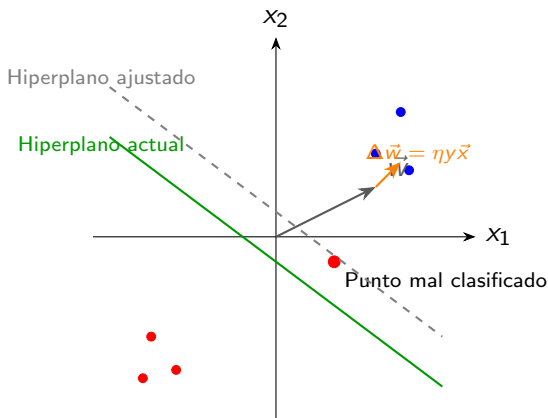
Por tanto, la actualización se realiza como:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta y \mathbf{x}, \quad b \leftarrow b + \eta y$$

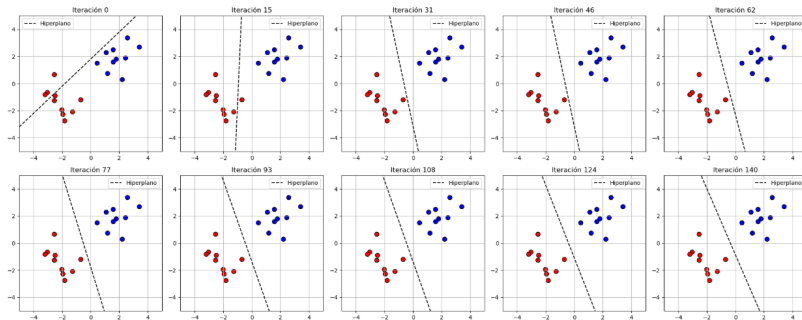
- ▶ Si el ejemplo está bien clasificado, no se actualiza.

Interpretación Geométrica.

- ▶ La regla de actualización desplaza el hiperplano hacia el ejemplo mal clasificado (el punto debe ser azul).
- ▶ Esto ajusta los parámetros para que el ejemplo quede del lado correcto en la siguiente iteración.



Ejemplo.



Ver código python.

Función de Pérdida Logarítmica (Log-Loss).

- ▶ Sea un problema de clasificación binaria con etiquetas:
 $y \in \{0, 1\}$.
- ▶ Se considera un modelo de regresión logística con término de sesgo:

$$\hat{y} = \sigma(z) = \frac{1}{1 + \exp(-z)}, \quad z = w^T x + b$$

- ▶ $x \in \mathbb{R}^d$: vector de características.
- ▶ $w \in \mathbb{R}^d$: vector de pesos.
- ▶ $b \in \mathbb{R}$: término de sesgo.
- ▶ $\sigma(z)$: función sigmoide.

Función de Pérdida Logarítmica (Log-Loss).

- ▶ Alternativa diferenciable ampliamente usada en clasificación binaria.
- ▶ Para una salida $\hat{y} = \sigma(z)$ con:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- ▶ La pérdida binaria (entropía cruzada⁶) se define como:

$$\mathcal{L}(w, b) = \mathcal{L}_{\log}(y, \hat{y}) = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

- ▶ $y \in \{0, 1\}$ es la etiqueta verdadera.
- ▶ $\hat{y} = \sigma(w^{\top}x + b)$ es la probabilidad predicha.
- ▶ Es convexa y derivable.

⁶En machine learning y deep learning, el logaritmo (log) es el logaritmo natural, es decir, $\ln$ (logaritmo en base e).

Verosimilitud del Dato.

La función de verosimilitud para un dato dado $(\mathbf{x}, y)$ está definida como:

$$P(y \mid \mathbf{x}; \mathbf{w}, b) = \hat{y}^y \cdot (1 - \hat{y})^{1-y}$$

donde $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ es la salida del modelo.

De este modo, se tienen dos casos:

- ▶ Si $y = 1$, entonces: $P(y = 1 \mid \mathbf{x}) = \hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$
- ▶ Si $y = 0$, entonces: $P(y = 0 \mid \mathbf{x}) = 1 - \hat{y}$

Esto significa que, dado un vector de características $\mathbf{x}$ y los parámetros del modelo w, b :

- ▶ el valor $\hat{y}$ representa la probabilidad condicional de que la etiqueta y sea 1 (clase positiva),
- ▶ y $1 - \hat{y}$ es la probabilidad correspondiente a la clase 0 (clase negativa).

Log-verosimilitud Negativa.

- ▶ Para definir una función de pérdida diferenciable, se toma el logaritmo negativo de la verosimilitud:

$$\mathcal{L}_{\log}(y, \hat{y}) = -\log P(y \mid \mathbf{x}; \mathbf{w}, b) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

- ▶ Sustituyendo $\hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + b)$, se obtiene:

$$\mathcal{L}_{\log}(y, \mathbf{x}; \mathbf{w}, b) = -\left[y \log(\sigma(\mathbf{w}^T \mathbf{x} + b)) + (1 - y) \log(1 - \sigma(\mathbf{w}^T \mathbf{x} + b))\right]$$

Esta es la llamada **pérdida logarítmica binaria**, utilizada en tareas de clasificación binaria.

Clasificador Softmax Binario (Regresión Logística).

- ▶ Se emplea pérdida de entropía cruzada (función a minimizar):

$$\mathcal{L} = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

Interpretación:

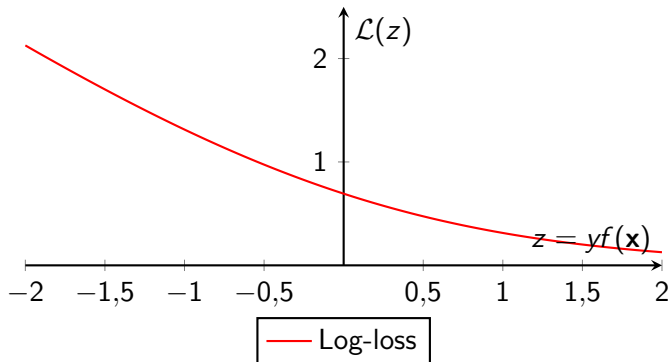
- ▶ Si $y = 1$, la pérdida es: $-\log(\hat{y})$
que es pequeña cuando $\hat{y} \approx 1$ y crece sin límite si $\hat{y} \rightarrow 0$.
- ▶ Si $y = 0$, la pérdida es: $-\log(1 - \hat{y})$
que es pequeña cuando $\hat{y} \approx 0$ y crece sin límite si $\hat{y} \rightarrow 1$.

De esta forma, se penalizan con mucha fuerza las predicciones incorrectas realizadas con alta confianza, ya que el término logarítmico tiende a $+\infty$ cuando la probabilidad predicha para la clase real es cercana a cero⁷.

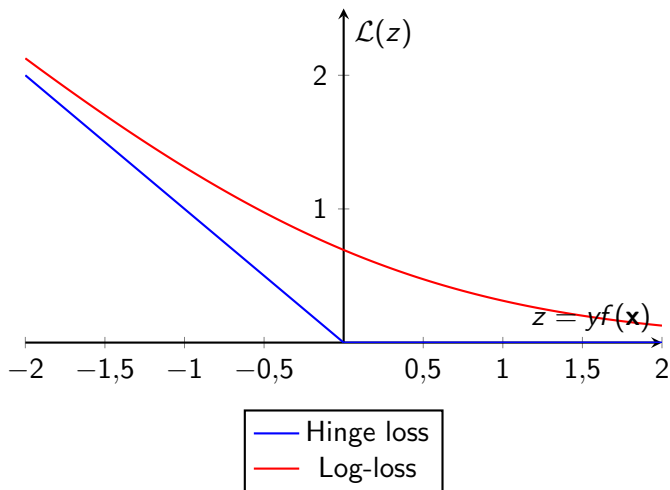
- ▶ Se entrena la neurona para minimizar $\mathcal{L}$ y clasificar.

⁷Ver: [Internet](#).

Gráfica de la Log-Loss.



Comparación de las funciones de pérdida.



Comparación de las funciones de pérdida.

- ▶ La pérdida hinge se activa solo cuando hay error, pero no es diferenciable.
- ▶ La log-loss es suave y permite optimización por gradiente descendente clásico.
- ▶ En clasificación binaria moderna, log-loss es preferida por su estabilidad y convexidad.

Minimización de la función de **pérdida logarítmica binaria**, por descenso del gradiente.

- ▶ La **derivada de la función sigmoide** es:

$$\begin{aligned}\frac{d\sigma(z)}{dz} &= \frac{d}{dz} \left(\frac{1}{1 + e^{-z}} \right) = \frac{d}{dz} ((1 + e^{-z})^{-1}) \\ &= -1 \cdot (1 + e^{-z})^{-2} \cdot (-e^{-z}) \\ &= \frac{e^{-z}}{(1 + e^{-z})^2}\end{aligned}$$

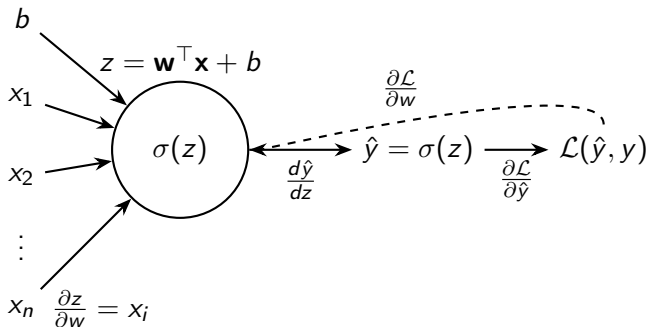
- ▶ Ahora, se expresa esta derivada en función de $\sigma(z)$, como:
$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z)).$$
- ▶ Finalmente, si se denota $\hat{y} = \sigma(z)$, se tiene: $\frac{d\sigma(z)}{dz} = \hat{y}(1 - \hat{y})$.

Este resultado es útil para aplicar la regla de la cadena.

Gradiente de la función de costo con respecto a los pesos w .

Se aplica la regla de la cadena:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{d\hat{y}}{dz} \cdot \frac{\partial z}{\partial w}$$



Sea la regla de la cadena:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{d\hat{y}}{dz} \cdot \frac{\partial z}{\partial w}$$

Entonces

- $\frac{\partial \mathcal{L}}{\partial \hat{y}}$: Derivada de la función de pérdida respecto a la salida del modelo.

Para la log-loss (entropía cruzada):

$$\mathcal{L}(\hat{y}, y) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Aplicando derivada parcial respecto a $\hat{y}$:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}} = - \left[y \cdot \frac{1}{\hat{y}} + (1 - y) \cdot \left(\frac{-1}{1 - \hat{y}} \right) \right]$$

$$\Rightarrow \boxed{\frac{\partial \mathcal{L}}{\partial \hat{y}} = - \left(\frac{y}{\hat{y}} - \frac{1 - y}{1 - \hat{y}} \right)}$$

- $\frac{d\hat{y}}{dz}$: Derivada de la función de activación (sigmoide):

$$\frac{d\hat{y}}{dz} = \frac{d\sigma(z)}{dz} = \hat{y}(1 - \hat{y})$$

- $\frac{\partial z}{\partial \mathbf{w}}$: Derivada de la función lineal respecto a los pesos:

$$z = \mathbf{w}^\top \mathbf{x} + b \quad \Rightarrow \quad \frac{\partial z}{\partial \mathbf{w}} = \mathbf{x}$$

Combinando todo:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \left(- \left(\frac{y}{\hat{y}} - \frac{1-y}{1-\hat{y}} \right) \right) \cdot \hat{y}(1 - \hat{y}) \cdot \mathbf{x}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = (\hat{y} - y)\mathbf{x}$$

Gradiente de la función de costo con respecto al sesgo b .

- ▶ Dado que $z = w^T x + b$, se tiene:

$$\frac{\partial z}{\partial b} = 1$$

- ▶ Entonces, el gradiente respecto al sesgo es:

$$\boxed{\frac{\partial \mathcal{L}}{\partial b} = (\hat{y} - y)}$$

Resumen: gradientes de la función Log-loss.

$$\boxed{\frac{\partial \mathcal{L}}{\partial w} = (\hat{y} - y)\mathbf{x}} \quad y \quad \boxed{\frac{\partial \mathcal{L}}{\partial b} = \hat{y} - y}$$

- ▶ Estos gradientes son simples y eficientes de calcular.
- ▶ Son ampliamente usados en tareas de clasificación binaria.

La función de costo se minimiza iterativamente:

$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}$$

Proceso de Entrenamiento:

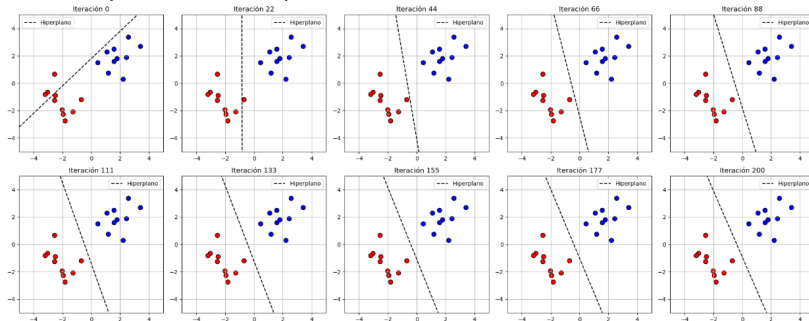
1. Inicializar los pesos $\mathbf{w}$ y el sesgo b aleatoriamente.
2. Repetir hasta convergencia:
 - 2.1 Para cada ejemplo $(\mathbf{x}^{(n)}, y^{(n)})$:

- ▶ Calcular la salida del perceptrón.
- ▶ Si hay error, actualizar los parámetros:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(y^{(n)} - \hat{y}^{(n)})\mathbf{x}^{(n)}$$

$$b \leftarrow b + \eta(y^{(n)} - \hat{y}^{(n)})$$

Ejemplo.



Ver código python.

Regularización.

- ▶ Para evitar el **sobreajuste**, se puede aplicar regularización.
- ▶ La regularización L2 (también llamada weight decay) penaliza pesos grandes:

$$\mathcal{R}_{L2}(\mathbf{w}) = \frac{\lambda}{2} \|\mathbf{w}\|^2 = \frac{\lambda}{2} \sum_j w_j^2$$

Con λ : el coeficiente de regularización.

- ▶ La **función de costo regularizada** se convierte en:

$$\mathcal{J}(\mathbf{w}, b) = \sum_{n=1}^N \mathcal{L}(\mathbf{w}, b; \mathbf{x}^{(n)}, y^{(n)}) + \mathcal{R}_{L2}(\mathbf{w})$$

Interpretación Biológica de la Regularización.

- ▶ En clasificación, es común añadir una penalización por complejidad:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{entropía o hinge}} + \lambda \|\mathbf{w}\|^2$$

- ▶ En términos biológicos, esta regularización puede interpretarse como:
 - ▶ **Olvido gradual:** los pesos sinápticos tienden a decrecer con el tiempo.
 - ▶ **Estabilidad:** evita que una neurona dependa excesivamente de pocas entradas.

Pérdida total:

Se añade término de regularización a la pérdida base:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{base}} + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

- Para Hinge loss:

$$\mathcal{L}_{\text{base}} = \max(0, 1 - yz), \quad \text{con } z = \mathbf{w}^T \mathbf{x}$$

- Para Log-loss:

$$\mathcal{L}_{\text{base}} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Hinge Loss con Regularización L_2 .

- Pérdida total:

$$\mathcal{L}_{\text{total}} = \underbrace{\max(0, 1 - y \cdot (\mathbf{w}^\top \mathbf{x}))}_{\text{Hinge loss}} + \underbrace{\frac{\lambda}{2} \|\mathbf{w}\|^2}_{\text{Regularización } L_2}$$

- Gradiente:

$$\nabla_{\mathbf{w}} \mathcal{L}_{\text{total}} = \begin{cases} -y\mathbf{x} + \lambda\mathbf{w}, & \text{si } 1 - y \cdot (\mathbf{w}^\top \mathbf{x}) > 0 \\ \lambda\mathbf{w}, & \text{si } 1 - y \cdot (\mathbf{w}^\top \mathbf{x}) \leq 0 \end{cases}$$

- Regla de actualización de pesos:

$$\mathbf{w} \leftarrow \begin{cases} \mathbf{w} - \eta(-y\mathbf{x} + \lambda\mathbf{w}), & \text{si violación del margen} \\ \mathbf{w} - \eta\lambda\mathbf{w}, & \text{si margen respetado} \end{cases}$$

- Si $y \cdot (\mathbf{w}^\top \mathbf{x}) < 1$: ejemplo mal clasificado o en el margen.
- Si $y \cdot (\mathbf{w}^\top \mathbf{x}) \geq 1$: ejemplo bien clasificado y fuera del margen.

Log-loss con Regularización L_2 .

- Pérdida total:

$$\mathcal{L}_{\text{total}} = \underbrace{-[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]}_{\text{Log-loss}} + \underbrace{\frac{\lambda}{2} \|\mathbf{w}\|^2}_{\text{Regularización } L_2}$$

- Recordando que: $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$
- Gradiente con respecto a $\mathbf{w}$:

$$\nabla_{\mathbf{w}} \mathcal{L}_{\text{total}} = (\hat{y} - y) \cdot \mathbf{x} + \lambda \mathbf{w}$$

- Regla de actualización de pesos:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta [(\hat{y} - y) \cdot \mathbf{x} + \lambda \mathbf{w}]$$

- El término $(\hat{y} - y)$ proviene de la derivada de la log-loss.
- La regularización L_2 empuja los pesos hacia cero para evitar sobreajuste.
- η : tasa de aprendizaje. λ : coeficiente de regularización.

Ejemplo en Python: El perceptrón como clasificador.

Objetivo

- ▶ Aplicar la regla de perceptrón actualizando pesos y sesgo minimizando funciones de costo.

$$w_j \leftarrow w_j + \eta(y^{(i)} - \hat{y}^{(i)})x_j^{(i)}, \quad b \leftarrow b + \eta(y^{(i)} - \hat{y}^{(i)})$$

- ▶ **Ver Notebook.**

Importancia de las Funciones de Activación.

Las funciones de activación **introducen no linealidad** en las redes neuronales, permitiendo que estas modelen funciones complejas.

- ▶ Se aplican a la salida de cada neurona.
- ▶ Permiten decidir si una neurona se activa o no.
- ▶ Su elección afecta la convergencia y desempeño del entrenamiento.

Función Lineal.

$$f(x) = x$$

- ▶ No transforma la entrada: salida igual a la entrada.
- ▶ Se utiliza ocasionalmente en la capa de salida para regresión.

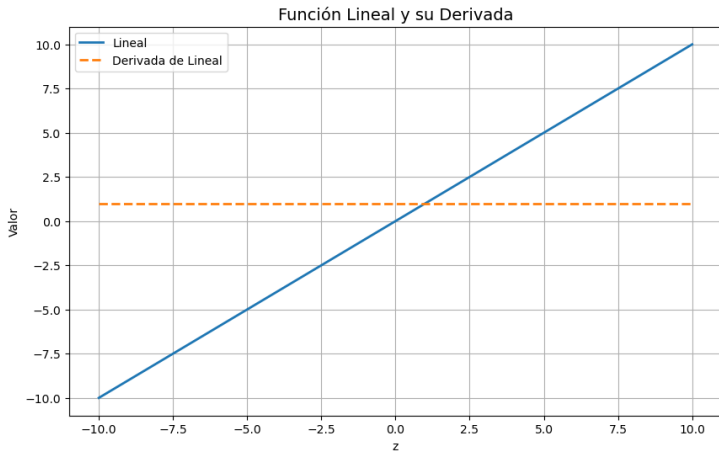
Derivada:

$$f'(x) = 1$$

Problemas de la Función Lineal:

- ▶ **No introduce no linealidad:** múltiples capas equivalen a una sola transformación lineal.
- ▶ **Capacidad limitada:** no permite modelar relaciones complejas o no lineales.
- ▶ **No útil en capas ocultas:** impide el aprendizaje profundo efectivo.

Función Lineal:



Función Sigmoide.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- ▶ Comprime valores reales al rango $[0, 1]$.
- ▶ Utilizada históricamente como modelo de tasa de activación de una neurona.

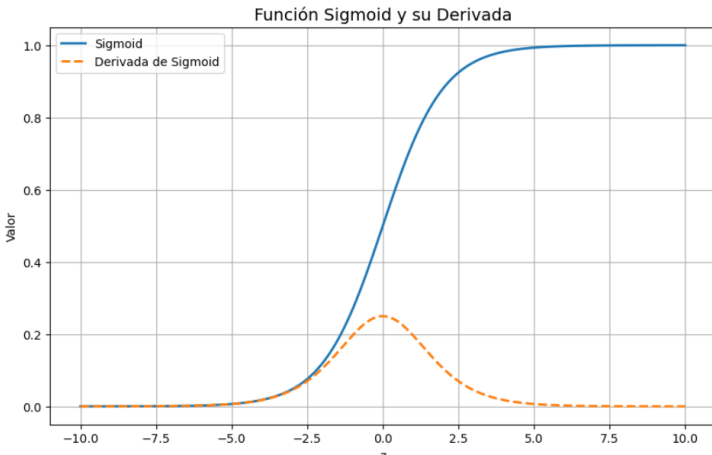
Derivada:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

Problemas de la Sigmoide:

- ▶ **Saturación y gradientes muertos:** para valores extremos, la derivada es casi cero.
- ▶ **No centrada en cero:** puede causar oscilaciones en el descenso de gradiente.
- ▶ **Obsoleta en redes profundas modernas.**

Función Sigmoide:



Función Tangente Hiperbólica (Tanh).

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- ▶ Comprime los valores reales al rango $[-1, 1]$.
- ▶ Centrada en cero: salida simétrica respecto al origen.
- ▶ Útil para acelerar la convergencia del entrenamiento.

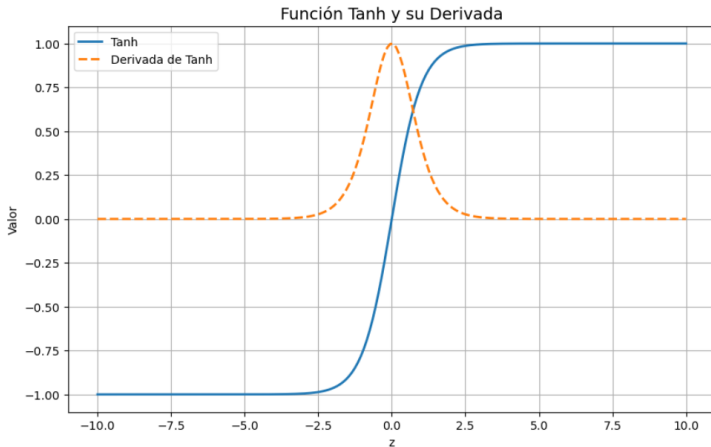
Derivada:

$$\tanh'(x) = 1 - \tanh^2(x)$$

Problemas de Tanh:

- ▶ **Saturación:** para valores extremos, el gradiente tiende a cero.
- ▶ **Vanishing gradient:** puede dificultar el entrenamiento en redes profundas.

Función Tanh:



Función ReLU (Rectified Linear Unit).

$$\text{ReLU}(x) = \max(0, x)$$

- ▶ No comprime los valores positivos: $x > 0 \Rightarrow \text{ReLU}(x) = x$.
- ▶ Introduce no linealidad sin saturación en valores positivos.
- ▶ Computacionalmente eficiente y ampliamente usada en redes profundas.

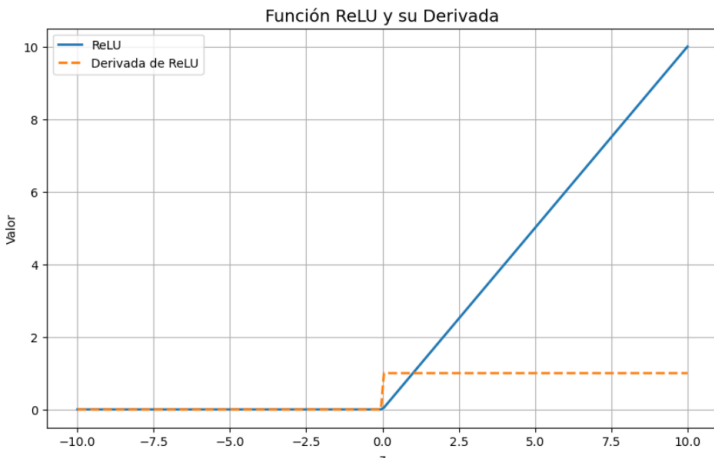
Derivada:

$$\text{ReLU}'(x) = \begin{cases} 1, & \text{si } x > 0 \\ 0, & \text{si } x \leq 0 \end{cases}$$

Problemas de ReLU:

- ▶ **Neuronas muertas:** si $x \leq 0$ de forma persistente, la neurona deja de aprender.
- ▶ **No diferenciable en $x = 0$,** aunque esto no es crítico.

Función ReLU:



Función Leaky ReLU.

$$\text{LeakyReLU}(x) = \begin{cases} x, & \text{si } x > 0 \\ \alpha x, & \text{si } x \leq 0 \end{cases} \quad \text{con } \alpha \ll 1$$

- ▶ Introduce una pendiente pequeña en la región negativa ($\alpha \approx 0,01$).
- ▶ Evita el problema de neuronas muertas de la ReLU.
- ▶ Retiene la simplicidad computacional.

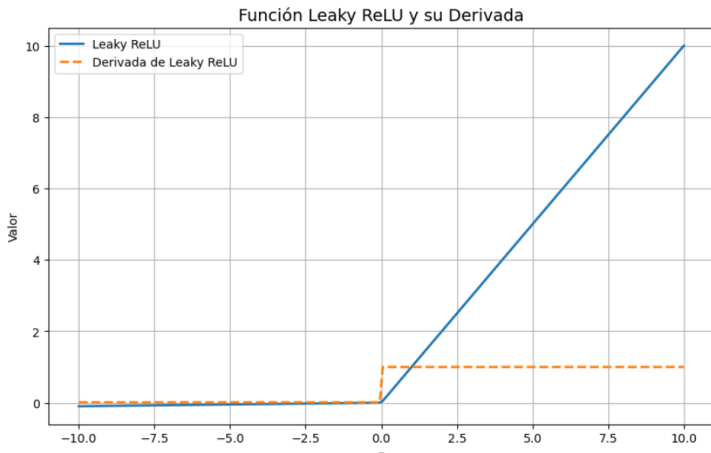
Derivada:

$$\text{LeakyReLU}'(x) = \begin{cases} 1, & \text{si } x > 0 \\ \alpha, & \text{si } x \leq 0 \end{cases}$$

Ventajas de Leaky ReLU:

- ▶ Mejora la propagación del gradiente en redes profundas.
- ▶ Puede aprender activaciones más robustas que ReLU.

Función Leaky ReLU:



Función Maxout.

$$\text{Maxout}(x) = \max(w_1^T x + b_1, w_2^T x + b_2)$$

- ▶ Generaliza ReLU y Leaky ReLU al tomar el máximo entre múltiples funciones lineales.
- ▶ No tiene una forma fija, sino que depende de parámetros entrenables.
- ▶ Puede aproximar cualquier función convexa por partes.

Derivada:

- ▶ No tiene una expresión cerrada general.
- ▶ Durante el entrenamiento, el gradiente se propaga solo por el argumento máximo.

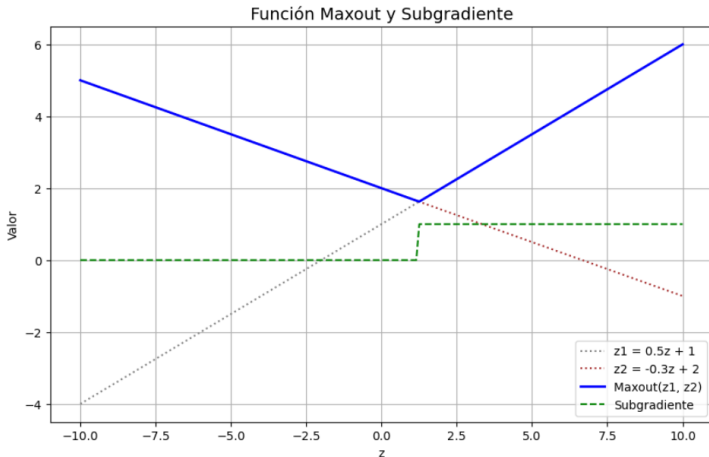
Ventajas de Maxout:

- ▶ Permite representar funciones más complejas que ReLU.
- ▶ Se adapta bien al dropout, al no depender de saturación.

Desventajas:

- ▶ Aumenta significativamente el número de parámetros.
- ▶ Requiere más memoria y cómputo.

Función Maxout:



Función Softmax.

Dada una entrada vectorial $z = [z_1, z_2, \dots, z_K]$, se define:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad \text{para } i = 1, \dots, K$$

- ▶ Convierte un vector de valores reales en una distribución de probabilidad.
- ▶ Comúnmente utilizada en la **capa de salida de clasificadores** multiclase.
- ▶ Garantiza que la suma de todas las salidas sea igual a 1.

Derivada (Jacobiano):

$$\frac{\partial \text{Softmax}(z_i)}{\partial z_j} = \text{Softmax}(z_i)(\delta_{ij} - \text{Softmax}(z_j))$$

Ventajas de Softmax:

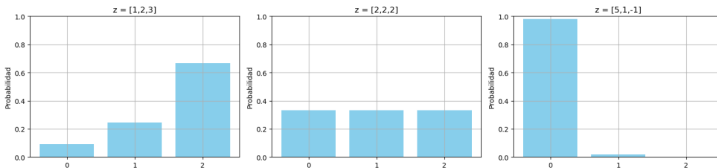
- ▶ Interpretable como probabilidades.
- ▶ Se usa junto con la pérdida **log-loss** (entropía cruzada).

Desventajas:

- ▶ Sensible a valores grandes: puede generar salidas numéricamente inestables si no se normaliza.

Función Softmax:

Salida de la Función Softmax para Diferentes Vectores z



Función Swish.

$$\text{Swish}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

- ▶ Función suave y no monótona.
- ▶ Propuesta por investigadores de Google como alternativa a ReLU.
- ▶ Combina la identidad con la función sigmoide de forma multiplicativa.

Derivada:

$$\text{Swish}'(x) = \sigma(x) + x \cdot \sigma'(x) = \sigma(x) + x \cdot \sigma(x)(1 - \sigma(x))$$

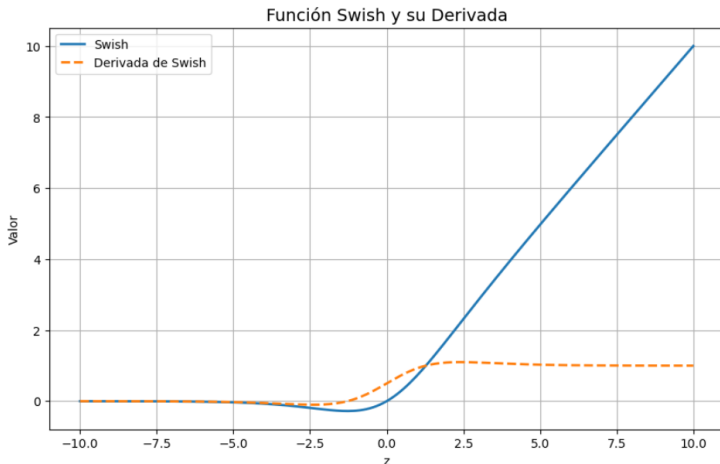
Ventajas de Swish:

- ▶ Supera a ReLU en varias tareas de visión y NLP.
- ▶ Suaviza el flujo de gradientes y evita saturación abrupta.
- ▶ Mejora la generalización y estabilidad en redes profundas.

Desventajas:

- ▶ Ligeramente más costosa computacionalmente que ReLU.

Función Swish:



Función GELU (Gaussian Error Linear Unit).

$$\text{GELU}(x) = x \cdot \Phi(x)$$

donde.

- ▶ $\Phi(x) = \frac{1}{2} \left[1 + \text{erf} \left(\frac{x}{\sqrt{2}} \right) \right]$

- ▶ $\text{erf}(x)$ es la función error (error function):

$$\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$$

- ▶ Activa la neurona de forma suave y probabilística.
- ▶ Es una aproximación a la multiplicación por la probabilidad de activación bajo una distribución normal.

Derivada exacta:

$$\frac{d}{dx} \text{GELU}(x) = \Phi(x) + x \cdot \phi(x) \quad \text{con } \phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$$

Aproximación práctica (Hendrycks & Gimpel):

$$\text{GELU}(x) \approx 0,5x \left(1 + \tanh \left[\sqrt{\frac{2}{\pi}}(x + 0,044715x^3) \right] \right)$$

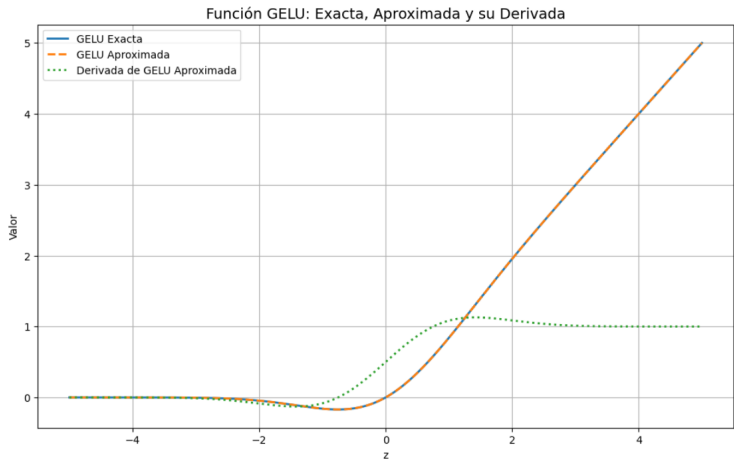
Ventajas:

- ▶ Mejora la regularización en redes profundas.
- ▶ Supera a ReLU en tareas de NLP y visión.

Desventajas:

- ▶ Computacionalmente más costosa.
- ▶ Derivada compleja en su forma exacta.

Función GELU:



Comparación de Funciones de Activación.

Función	Fórmula	Rango	Centrada en 0	Saturación
Lineal	$f(x) = x$	$(-\infty, \infty)$	Sí	No
Sigmoide	$\frac{1}{1+e^{-x}}$	$[0, 1]$	No	Si
Tanh	$\frac{e^x - e^{-x}}{e^x + e^{-x}}$	$[-1, 1]$	Si	Si
ReLU	$\max(0, x)$	$[0, \infty)$	No	No
Leaky ReLU	$\max(\alpha x, x)$	$(-\infty, \infty)$	No	No
Maxout	$\max(w_1^T x + b_1, w_2^T x + b_2)$	Depende	Si (posible)	No
Softmax	$\frac{e^{x_j}}{\sum_j e^{x_j}}$	$(0, 1)$	No	Si
Swish	$x \cdot \sigma(x)$	$(-\infty, \infty)$	Parcial	Si
GELU	$x \cdot \Phi(x)$	$(-\infty, \infty)$	Si	Si

Función	Derivable	Aplicaciones
Lineal	Sí	Capa de salida en regresión
Sigmoide	Si	Histórica, salida binaria
Tanh	Si	Capas ocultas, redes clásicas
ReLU	Parcial	Deep learning, visión
Leaky ReLU	Si	Reducción de neuronas muertas
Maxout	Si (por partes)	Funciones arbitrarias, con dropout
Softmax	Si	Capa de salida multiclase
Swish	Si	Redes profundas, reemplazo de ReLU
GELU	Si	Transformers (BERT, etc.)

Clasificación Multiclase con Modelos Lineales.

- ▶ Tras abordar la clasificación binaria (dos categorías), se presenta el desafío de la **clasificación multiclase**.
- ▶ Esta tarea consiste en asignar una instancia a una de *múltiples* clases posibles (ej. reconocimiento de dígitos, diagnóstico de enfermedades, etc).
- ▶ Aunque un modelo lineal simple (como el Perceptrón) define un separador binario (un hiperplano), existen estrategias para extender su uso a escenarios multiclase:
 - ▶ Uno contra Uno (One-vs-One / OvO).
 - ▶ Uno contra el Resto (One-vs-Rest / OvR).
 - ▶ Perceptrón Multiclase.
 - ▶ Generalización directa: Modelos como la **Regresión Softmax** (o Regresión Logística Multiclase).

Clasificación One-versus-One (OvO).

- ▶ En problemas de clasificación multiclase, el objetivo es asignar una muestra a una entre K clases posibles.
- ▶ El enfoque One-vs-One construye un clasificador binario para cada par de clases.
- ▶ Para un problema con K clases, se entrenan $\frac{K(K-1)}{2}$ clasificadores.
- ▶ Cada clasificador decide entre dos clases, ignorando las demás.
- ▶ La clase predicha es la que sea predicha por el mayor número de clasificadores.

- ▶ Cada clasificador binario es un modelo lineal:

$$h_{ij}(\mathbf{x}) = \mathbf{w}_{ij}^T \mathbf{x} + b_{ij}$$

- ▶ **La función de activación es lineal** (identidad).
- ▶ La predicción entre clases i y j se toma como:

$$\hat{y}_{ij} = \begin{cases} i & \text{si } h_{ij}(\mathbf{x}) > 0 \\ j & \text{si } h_{ij}(\mathbf{x}) < 0 \end{cases}$$

Notación:

- ▶ y : clase verdadera (etiqueta real).
- ▶ $\hat{y}$: clase predicha.
- ▶ $h_{ij}(\mathbf{x})$: salida cruda del clasificador binario para clases i y j .

- ▶ En cada clasificador (i, j) se codifican las etiquetas:

$$y \in \{+1, -1\}, \quad y_{ij} = \begin{cases} +1 & \text{si la clase verdadera es } i \\ -1 & \text{si es } j \end{cases}$$

- ▶ Se comete **error** si la salida cruda no tiene el signo esperado:

$$\text{Error si: } y \cdot f_{ij}(\mathbf{x}) \leq 0$$

- ▶ **Función de pérdida común:** hinge loss (tipo perceptrón o SVM):

$$\mathcal{L}_{ij} = \max(0, -y \cdot h_{ij}(\mathbf{x}))$$

Actualización de Pesos.

- ▶ Cada clasificador binario (i, j) se entrena de manera independiente.
- ▶ Si hay error $(y \cdot f_{ij}(\mathbf{x}) \leq 0)$, se actualiza:

$$\mathbf{w}_{ij} \leftarrow \mathbf{w}_{ij} + \eta \cdot y \cdot \mathbf{x}$$

$$b_{ij} \leftarrow b_{ij} + \eta \cdot y$$

- ▶ Donde η es la tasa de aprendizaje.

Función de Decisión.

- ▶ Cada clasificador emite un voto a favor de una de las dos clases involucradas.
- ▶ Se cuentan los votos obtenidos por cada clase:

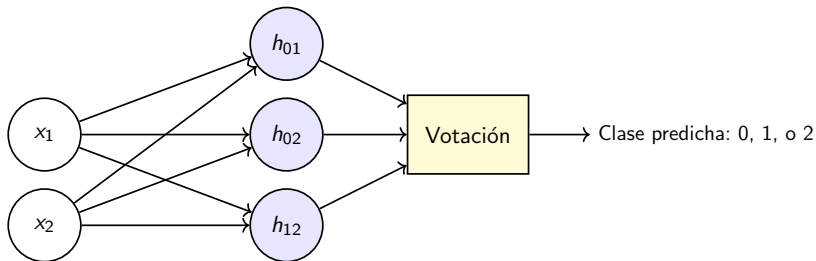
V_k = número de veces que la clase k fue elegida

- ▶ La clase con más votos se asigna como predicción final:

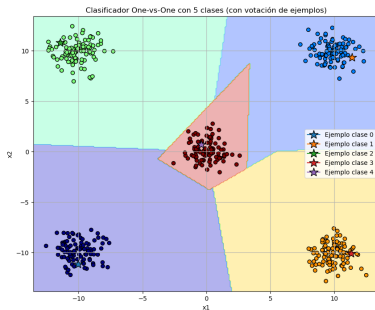
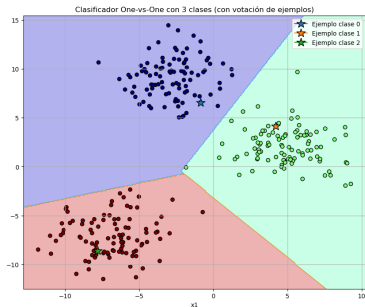
$$\hat{y} = \arg \max_k V_k$$

Ejemplo con tres clases:

- ▶ Sea un problema con clases: 0, 1, 2.
- ▶ Se entrenan 3 clasificadores (perceptrones): h_{01} , h_{02} y h_{12} .
- ▶ Para una muestra $\mathbf{x}$, se obtienen las siguientes predicciones:
 - ▶ $h_{01}(\mathbf{x}) = +1$, entonces voto para clase 0
 - ▶ $h_{02}(\mathbf{x}) = -1$, entonces voto para clase 2
 - ▶ $h_{12}(\mathbf{x}) = -1$, entonces voto para clase 2
- ▶ El conteo de votos arroja que la clase predicha es: **clase 2**.



Ejemplo visual.



Ventajas y Desventajas del Método OvO.

▶ Ventajas:

- ▶ Más eficiente que OvR cuando el número de muestras por clase es grande.
- ▶ Cada clasificador trabaja con un subconjunto reducido de datos.
- ▶ Puede ofrecer mejor precisión en algunos casos.

▶ Desventajas:

- ▶ El número de clasificadores crece cuadráticamente con K .
- ▶ La votación puede resultar ambigua si hay empate.

Ver código python.

Clasificación One-versus-Rest (OvR).

- ▶ El enfoque One-vs-Rest entrena un clasificador binario para cada clase k , contra todas las demás.
- ▶ Para un problema con K clases, se entrenan K clasificadores independientes.
- ▶ Cada clasificador decide si una muestra pertenece o no a su clase.

- ▶ Cada clasificador binario es un modelo lineal:

$$h_k(\mathbf{x}) = \mathbf{w}_k^\top \mathbf{x} + b_k$$

- ▶ **La función de activación es lineal** (identidad).
- ▶ La predicción se toma como:

$$\hat{y} = \arg \max_k h_k(\mathbf{x})$$

Notación:

- ▶ y : clase verdadera (etiqueta real).
- ▶ $\hat{y}$: clase predicha.
- ▶ $h_k(\mathbf{x})$: salida cruda del clasificador binario para la clase k .

- ▶ En cada clasificador k , se codifican las etiquetas:

$$y \in \{+1, -1\}, \quad y_k = \begin{cases} +1 & \text{si la clase verdadera es } k \\ -1 & \text{si es una clase distinta} \end{cases}$$

- ▶ Se comete **error** si la salida cruda no tiene el signo esperado:

$$\text{Error si: } y \cdot h_k(\mathbf{x}) \leq 0$$

- ▶ **Función de pérdida común:** hinge loss:

$$\mathcal{L}_k = \max(0, -y \cdot h_k(\mathbf{x}))$$

Actualización de Pesos.

- ▶ Cada clasificador binario k se entrena de forma independiente.
- ▶ Si hay error ($y \cdot h_k(\mathbf{x}) \leq 0$), se actualiza:

$$\mathbf{w}_k \leftarrow \mathbf{w}_k + \eta \cdot y \cdot \mathbf{x}$$

$$b_k \leftarrow b_k + \eta \cdot y$$

- ▶ Donde η es la tasa de aprendizaje.

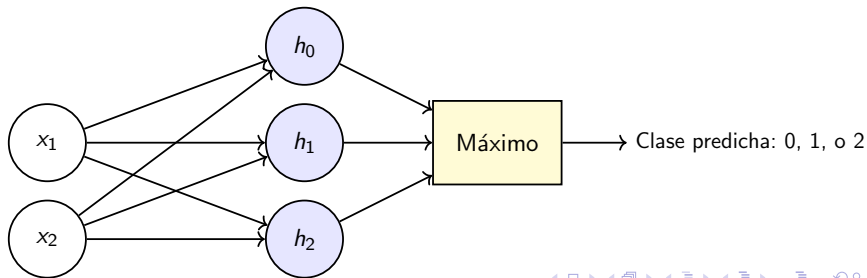
Función de Decisión.

- ▶ Se evalúan todas las salidas $h_k(\mathbf{x})$, una por clase.
- ▶ La clase predicha es la que tiene la mayor activación:

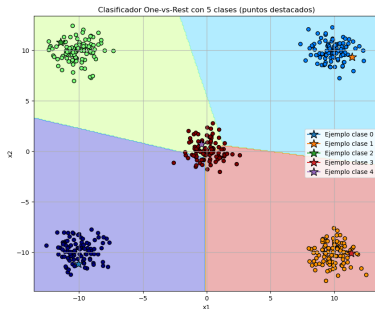
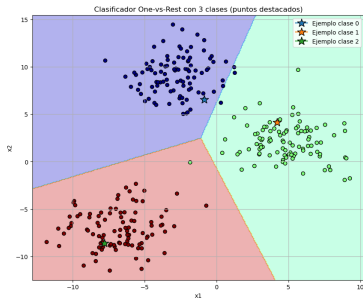
$$\hat{y} = \arg \max_k h_k(\mathbf{x})$$

Ejemplo con tres clases:

- ▶ Sea un problema con clases: 0, 1, 2.
- ▶ Se entrenan 3 clasificadores: h_0 , h_1 , h_2 .
- ▶ Para una muestra $\mathbf{x}$, se obtienen las siguientes salidas:
 - ▶ $s_0(\mathbf{x}) = -0,8$
 - ▶ $s_1(\mathbf{x}) = +1,2$
 - ▶ $s_2(\mathbf{x}) = +0,3$
- ▶ La clase predicha es: **clase 1**, ya que tiene la puntuación más alta.



Ejemplo visual.



Ventajas y Desventajas del Método OvR.

▶ Ventajas:

- ▶ Solo requiere entrenar K clasificadores, lo que es más eficiente que OvO cuando K es grande.
- ▶ Fácil de implementar con clasificadores binarios.
- ▶ Escalable a un gran número de clases.

▶ Desventajas:

- ▶ Puede haber regiones del espacio donde múltiples clasificadores predicen positivo.
- ▶ En presencia de datos desbalanceados, algunos clasificadores pueden dominar.

Ver código python.

Perceptrón Multiclase.

- ▶ Se entrena un único modelo con K salidas, una por clase.
- ▶ Cada clase k tiene un vector de pesos $\mathbf{w}_k \in \mathbb{R}^d$.
- ▶ La salida del modelo es un conjunto de puntuaciones lineales:

$$h_k(\mathbf{x}) = \mathbf{w}_k^\top \mathbf{x}$$

Función de Activación: lineal (identidad).

- ▶ Para una muestra de entrada $\mathbf{x} \in \mathbb{R}^d$, se calcula:

$$f(\mathbf{x}) = \begin{bmatrix} h_1(\mathbf{x}) \\ h_2(\mathbf{x}) \\ \vdots \\ h_K(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \mathbf{w}_1^\top \mathbf{x} \\ \mathbf{w}_2^\top \mathbf{x} \\ \vdots \\ \mathbf{w}_K^\top \mathbf{x} \end{bmatrix}$$

- ▶ Se predice la clase con mayor activación:

$$\hat{y} = \arg \max_k h_k(\mathbf{x})$$

Notación:

- ▶ y : clase verdadera.
- ▶ $\hat{y}$: clase predicha.

Error de Clasificación.

- ▶ Se comete error si:

$$\hat{y} \neq y \quad \text{o equivalentemente:} \quad h_y(\mathbf{x}) \leq h_k(\mathbf{x}) \text{ para alguna } k \neq y$$

- ▶ Es decir, cuando la salida correspondiente a la clase verdadera $h_y(\mathbf{x})$ **no es la mayor de todas**, entonces el modelo falló.

Función de Pérdida.

- ▶ Puede usarse una pérdida tipo perceptrón multiclase:

$$\mathcal{L}(\mathbf{x}, y) = \max_{k \neq y} [h_k(\mathbf{x}) - h_y(\mathbf{x})]$$

Actualización de Pesos (Regla del Perceptrón Multiclase).

- ▶ Si se comete error, se actualizan los pesos:

$$\mathbf{w}_y \leftarrow \mathbf{w}_y + \eta \cdot \mathbf{x} \quad \mathbf{w}_{\hat{y}} \leftarrow \mathbf{w}_{\hat{y}} - \eta \cdot \mathbf{x}$$

- ▶ Solo dos vectores de pesos se modifican por iteración:
 - ▶ $\mathbf{w}_y \leftarrow \mathbf{w}_y + \eta \mathbf{x}$, refuerza el vector de pesos de la **clase verdadera** y , para que su producto escalar con $\mathbf{x}$ **sea más grande** la próxima vez.
Esto aumenta la puntuación de la clase correcta.
 - ▶ $\mathbf{w}_{\hat{y}} \leftarrow \mathbf{w}_{\hat{y}} - \eta \mathbf{x}$, penaliza el vector de pesos de la **clase predicha erróneamente** $\hat{y}$, para que su puntuación con respecto a $\mathbf{x}$ **sea menor** en el futuro.
- ▶ η : tasa de aprendizaje.

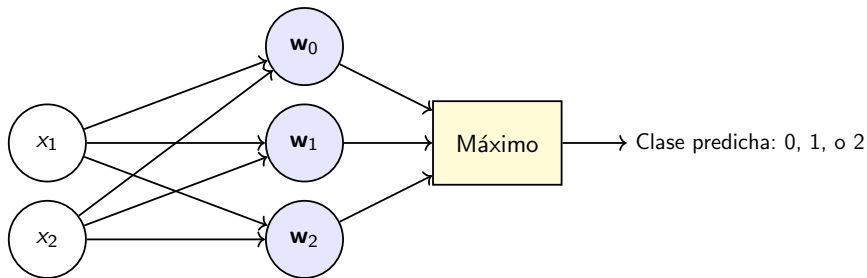
Función de Decisión Final.

- ▶ La clase predicha es la que obtiene mayor puntuación:

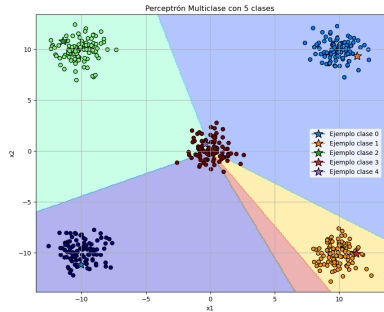
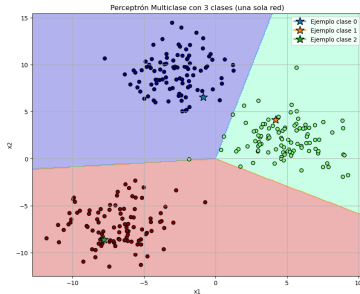
$$\hat{y} = \arg \max_k \left(\mathbf{w}_k^\top \mathbf{x} \right)$$

Ejemplo con tres clases:

- ▶ Sea un problema con clases: 0, 1, 2.
- ▶ Se entrenan vectores de pesos $\mathbf{w}_0$, $\mathbf{w}_1$, $\mathbf{w}_2$.
- ▶ Para una muestra $\mathbf{x}$, se calculan las salidas:
 - ▶ $s_0(\mathbf{x}) = -1,3$
 - ▶ $s_1(\mathbf{x}) = +2,1$
 - ▶ $s_2(\mathbf{x}) = +0,7$
- ▶ La clase predicha es: **clase 1**, al tener la puntuación más alta.



Ejemplo visual.



Ventajas y Desventajas del Perceptrón Multiclase

▶ Ventajas:

- ▶ Entrenamiento directo y simultáneo de todos los clasificadores.
- ▶ Evita ambigüedad entre clasificadores (como en OvR).
- ▶ Simplicidad en la predicción: máximo entre salidas.

▶ Desventajas:

- ▶ **No apto para problemas no linealmente separables.**
- ▶ La convergencia no está garantizada si los datos no son separables.
- ▶ Sensible a la escala de entrada y al orden del entrenamiento.

Ver código python.

Clasificación con Regresión Softmax.

- ▶ En problemas con K clases, se entrena un único modelo con K salidas lineales.
- ▶ Se calcula una probabilidad para cada clase mediante la función softmax.
- ▶ Este enfoque proporciona una salida probabilística que suma 1.

Función de Activación: Softmax

$$\hat{y}_k = \frac{e^{z_k(\mathbf{x})}}{\sum_{j=1}^K e^{z_j(\mathbf{x})}} \quad \text{para } k = 1, \dots, K$$

- ▶ $z_k(\mathbf{x}) = \mathbf{w}_k^\top \mathbf{x} + b_k$ es la salida cruda (logit) para la clase k .
- ▶ La salida $\hat{y}_k$, resultado de pasar z_k por la **función de activación** $\text{softmax}(z_k)$, representa la probabilidad predicha para la clase k .
- ▶ Se asegura que $\sum_k \hat{y}_k = 1$.

Función de Pérdida: Entropía Cruzada.

- ▶ Dado un conjunto de clases $\{1, \dots, K\}$, se definen los logits (salidas lineales) como:

$$z_k = \mathbf{w}_k^\top \mathbf{x} + b_k$$

- ▶ La función softmax convierte los logits en probabilidades:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

- ▶ La etiqueta real se codifica como vector **one-hot**:

$$y_k = \begin{cases} 1 & \text{si } k = \text{clase verdadera} \\ 0 & \text{si } k \neq \text{clase verdadera} \end{cases}$$

- ▶ Solo se penaliza la probabilidad de la clase verdadera.

- ▶ Se utiliza la pérdida de entropía cruzada para comparar la predicción con la verdad:

$$\mathcal{L}(\mathbf{x}, \mathbf{y}) = - \sum_{k=1}^K y_k \log(\hat{y}_k)$$

- ▶ Si $\mathbf{y}$ es one-hot, entonces:

$$\mathcal{L} = -\log(\hat{y}_y) \quad (\text{donde } y \text{ es la clase verdadera})$$

- ▶ Se desea calcular $\frac{\partial \mathcal{L}}{\partial z_j}$ para cada logit z_j .

Se usa la regla de la cadena:

$$\frac{\partial \mathcal{L}}{\partial z_i} = \sum_{k=1}^K \frac{\partial \mathcal{L}}{\partial \hat{y}_k} \cdot \frac{\partial \hat{y}_k}{\partial z_i}$$

Derivadas Intermedias.

1. Derivada de entropía cruzada:

$$\boxed{\frac{\partial \mathcal{L}}{\partial \hat{y}_k} = -\frac{y_k}{\hat{y}_k}}$$

2. Derivada de la softmax:

$$\frac{\partial \hat{y}_k}{\partial z_i} = \begin{cases} \hat{y}_k(1 - \hat{y}_k) & \text{si } k = i \\ -\hat{y}_k \hat{y}_i & \text{si } k \neq i \end{cases}$$

Forma unificada:

$$\boxed{\frac{\partial \hat{y}_k}{\partial z_i} = \hat{y}_k(\delta_{ki} - \hat{y}_i)} \quad \text{con } \delta_{ki} = \begin{cases} 1 & \text{si } k = i \\ 0 & \text{si } k \neq i \end{cases}$$

Gradiente de la Pérdida.

- ▶ Sustituyendo en la regla de la cadena:

$$\frac{\partial \mathcal{L}}{\partial z_i} = - \sum_{k=1}^K y_k \cdot (\delta_{ki} - \hat{y}_i)$$

- ▶ Separando los términos:

$$\frac{\partial \mathcal{L}}{\partial z_i} = -y_i(1 - \hat{y}_i) + \sum_{k \neq i} y_k \hat{y}_i = \hat{y}_i - y_i$$

- ▶ Resultado final:

$$\boxed{\frac{\partial \mathcal{L}}{\partial z_i} = \hat{y}_i - y_i}$$

► Interpretación del gradiente:

- Si $j = y$: se disminuye el error aumentando z_y .
- Si $j \neq y$: se penaliza cualquier clase incorrectamente favorecida.
- Este gradiente se propaga hacia atrás en redes neuronales profundas.
- Este gradiente es simple, eficiente y muy común en clasificación multiclase.

Actualización de Pesos

- ▶ Para minimizar la pérdida, se usa descenso por gradiente:

$$\mathbf{w}_k \leftarrow \mathbf{w}_k - \eta \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{w}_k}$$

- ▶ El gradiente de la pérdida para la clase k es:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}_k} = (\hat{y}_k - y_k) \mathbf{x}$$

- ▶ Este valor es positivo si la clase fue sobrestimada, y negativo si fue subestimada.

Función de Decisión

- ▶ Se predice la clase con mayor probabilidad:

$$\hat{y} = \arg \max_k \hat{y}_k$$

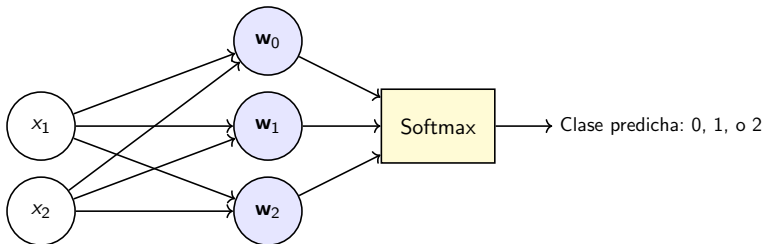
- ▶ Gracias a la softmax, las salidas son interpretables como probabilidades.
- ▶ Útil en tareas donde se requiere cuantificar la incertidumbre.

Notación:

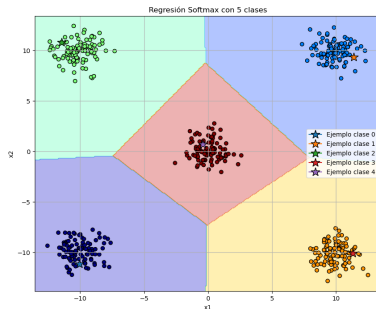
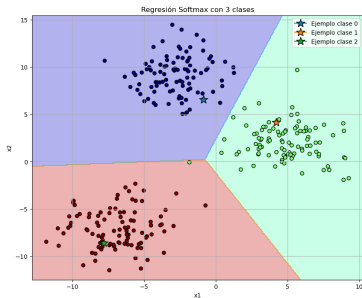
- ▶ y : etiqueta verdadera (one-hot o clase).
- ▶ $\hat{y}$: clase predicha (índice con máxima probabilidad).
- ▶ $\hat{y}_k$: probabilidad asignada a la clase k .

Ejemplo con tres clases:

- ▶ Sea un problema con clases: 0, 1, 2.
- ▶ Se entrenan vectores de pesos: $\mathbf{w}_0, \mathbf{w}_1, \mathbf{w}_2$.
- ▶ Para una muestra $\mathbf{x}$, con logits $z_0 = 1,0$, $z_1 = 2,0$ y $z_2 = 0,5$, se calculan las probabilidades:
 - ▶ $P(y = 0|\mathbf{x}) = \frac{e^{1,0}}{e^{1,0} + e^{2,0} + e^{0,5}} \approx 0,211$
 - ▶ $P(y = 1|\mathbf{x}) = \frac{e^{2,0}}{e^{1,0} + e^{2,0} + e^{0,5}} \approx 0,576$
 - ▶ $P(y = 2|\mathbf{x}) = \frac{e^{0,5}}{e^{1,0} + e^{2,0} + e^{0,5}} \approx 0,213$
- ▶ La clase predicha es: **clase 1**, por tener la mayor probabilidad.



Ejemplo visual.



Ventajas y Desventajas de la Regresión Softmax.

▶ **Ventajas:**

- ▶ Generalización directa y probabilística del problema multiclase.
- ▶ Permite interpretación probabilística de la salida.
- ▶ Entrenamiento con funciones suaves y diferenciables.

▶ **Desventajas:**

- ▶ Supone independencia condicional de clases.
- ▶ Más sensible al sobreajuste si K es grande y el conjunto de datos es pequeño.

Comparación de Enfoques Multiclase.

Aspecto	OvO (One-vs-One)	OvR (One-vs-Rest)	Perceptrón Multiclase	Regresión Softmax
Diferencia clave	Clasificadores entre pares de clases	Un clasificador por clase vs el resto	Un único modelo con K salidas lineales	Modelo probabilístico con salidas normalizadas
# Modelos	$\binom{K}{2} = \frac{K(K-1)}{2}$	K	1	1
Función de activación	Lineal: $z = \mathbf{w}^\top \mathbf{x}$	Lineal: $z = \mathbf{w}_k^\top \mathbf{x}$	Lineal: $z_k = \mathbf{w}_k^\top \mathbf{x}$	Softmax: $P(k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$
Función de pérdida	Hinge: $\max(0, 1 - y_{ij}z)$	Hinge: $\max(0, 1 - y_kz)$	Hinge multiclase: $\max_{j \neq y} (z_j - z_y + 1)$	Log-loss: $-\log P(y)$
Actualización pesos	$\mathbf{w}_{ij} \leftarrow \mathbf{w}_{ij} + y_{ij}\mathbf{x}$ si $e \leq 0$	$\mathbf{w}_k \leftarrow \mathbf{w}_k + y_k\mathbf{x}$ si $e \leq 0$	$\mathbf{w}_y \leftarrow \mathbf{w}_y + \mathbf{x}$, $\mathbf{w}_{\hat{y}} \leftarrow \mathbf{w}_{\hat{y}} - \mathbf{x}$ si $\hat{y} \neq y$	$\mathbf{w}_k \leftarrow \mathbf{w}_k - \eta \cdot \delta_k \cdot \mathbf{x}$
Función de decisión	Votación entre clasificadores	$\arg \max_k z_k$	$\arg \max_k z_k$	$\arg \max_k P(k)$
Salida probabilística	No	No	No	Sí
Interacción entre clases	No	No	Sí (por corrección conjunta)	Sí (en la función softmax)
Escalabilidad	Pobre si K grande	Buena	Buena	Buena
Separabilidad requerida	Lineal por pares	Lineal por clase vs resto	Lineal multiclase	No lineal (puede usar regularización)

Ejemplo con los cuatro enfoques.

Objetivo:

- ▶ Comprender el enfoque *Uno contra Uno*.
- ▶ Analizar el método *Uno contra el Resto*.
- ▶ Estudiar el funcionamiento del **Perceptrón Multiclase** como extensión directa del perceptrón binario.
- ▶ Explorar modelos de generalización directa para clasificación multiclase, como la **Regresión Softmax**.

- ▶ **Ver Notebook.**

Gracias por su atención.

